



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Grado en Ingeniería Informática

Trabajo Fin de Grado

**Booster: Una nueva plataforma de
videos de código abierto**

Autor: Samuel Caraballo Chichiraldi
Tutor: Fernando Pérez Costoya

Madrid, Junio 2026

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado
Grado en Ingeniería Informática

Título: Booster: Una nueva plataforma de videos de código abierto
Junio 2026

Autor: Samuel Caraballo Chichiraldi
Tutor: Fernando Pérez Costoya
Departamento de Arquitectura y Tecnología de Sistemas Informáticos
Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid

Resumen

El trabajo aborda el diseño y la implementación de una red social de vídeos en formato web llamada Booster. Se detallará el funcionamiento de la infraestructura de la plataforma así como las tecnologías utilizadas. Además, se hará una evaluación de las distintas opciones para procesar los vídeos: Bunny.net, Mux y AWS. También se documentará el código, las decisiones de diseño y se realizará una guía de despliegue.

El objetivo del proyecto es implementar un prototipo (Producto Mínimo Viable) y detallar cómo funcionan todas las partes de la plataforma de vídeos:

- autenticación
- arquitectura del procesamiento y transcodificación de vídeos
- despliegue,
- diseños de las interfaces de usuario
- optimizaciones
- riesgos de seguridad
- creación de comentarios
- notificaciones
- diseño del algoritmo de recomendación
- creación de comunidades
- carga de vídeos y edición de sus metadatos
- recuento de visitas
- calificaciones a los vídeos
- filtrado de los vídeos por categorías
- recuento de los seguidores de un usuario
- explicación del sistema de Boost y cómo funciona la moneda virtual (XP)
- diseño de la base de datos
- conexión entre front-end y back-end y explicación de las tecnologías usadas:

-
- Next.js
 - tRPC
 - NeonDB
 - DrizzleORM
 - SupaBase
 - TailwindCSS
 - ReactJS

Abstract

Abstract of the Final Degree Project. Maximum length: 2 pages.

Tabla de contenidos

1. Introducción	1
1.1. Motivación y necesidades del proyecto	1
1.2. Objetivos	3
1.3. Estructura de la memoria	3
2. Estado del Arte	7
2.1. Procesamiento, Transcodificación y Entrega de Video	7
2.1.1. FFmpeg	7
2.1.2. MPEG-DASH	9
2.1.3. HLS	10
2.1.4. Object Storage	11
2.1.5. CDN - Content Delivery Network	12
2.1.6. Colas de Procesamiento	14
2.2. Análisis de Proveedores	15
2.2.1. Amazon Web Services y solución propia	15
2.2.2. Mux.com	17
2.2.3. Bunny.net	17
3. Tecnologías Estudiadas	19
4. Requisitos del Sistema	21
5. Innovaciones	23
6. Diseño y Arquitectura	25
7. Desarrollo	27
7.1. Apartado 1 de capítulo 2	27
7.1.1. Sección 1 de apartado 1 de capítulo 2	27
7.1.2. Sección 2 de apartado 1 de capítulo 2	28
7.2. Apartado 2 de capítulo 2	28
7.3. Apartado 3 de capítulo 2	28
8. Pruebas, Validación y Riesgos de Seguridad Detectados	29
9. Conclusiones	31

TABLA DE CONTENIDOS

10. Análisis de Impacto	33
Bibliografía	35
Anexos	39
A. Primer anexo	39

Capítulo 1

Introducción

1.1. Motivación y necesidades del proyecto

Los servicios de video bajo demanda se encuentra en una fase de crecimiento continuo y altísima intensidad de uso, con YouTube, TikTok, Netflix y los Reels de Meta siendo las principales plataformas dominantes del sector. Esto es confirmado con los datos de Google [1] el cual reportó que los ingresos de YouTube por publicidad y suscripciones superaron los 60 mil millones de dólares en 2025, teniendo un alcance publicitario mensual de 2.53 mil millones de usuarios a inicios de 2025, como fue demostrado por DataReportal.

Asimismo, en abril de ese mismo año YouTube informó que la plataforma acumulaba más de 20 mil millones de videos subidos, que recibía más de 20 millones de videos nuevos al día, y que en 2024 promedió más de 100 millones de comentarios diarios y más de 3.5 mil millones de “likes” por día. Esto demuestra no sólo la escala, sino también la intensidad de uso de las plataformas de este estilo.

De forma similar, TikTok [2] confirmó que en 2025 ya superaba los 200 millones de usuarios mensuales en Europa. Por otro lado, estimaciones de eMarketer exponían el tiempo promedio que destinaba un usuario en la plataforma por día, con cifras que rondaban los 52 minutos diarios por usuario.

Netflix, por su parte, informó [3] que recibió más de 325 millones de membresías pagadas y 96 mil millones de horas vistas en el segundo semestre de 2025. Eso equivale a aproximadamente 522 millones de horas reproducidas por día durante ese semestre, lo cual es un signo claro de la persistencia de consumo y la alta inversión temporal que dedican los usuarios.

En Meta, la situación es similar. La empresa [4] reportó 3.58 mil millones de personas activas por día en su familia de aplicaciones en diciembre de 2025 y comunicó, adicionalmente, que en Estados Unidos el tiempo de visualización de video en Facebook creció más de 20 por ciento interanual, lo que refleja el consumo masivo de videos en línea y el dominio del video de formato corto basado en algoritmos que priorizan el contenido dinámico.

Capítulo 1. Introducción

Es importante destacar, el impacto medioambiental que tiene este consumo masivo de contenido en línea. Un estudio realizado por La Agencia Internacional de la Energía [5] estimó que los centros de datos consumieron alrededor de 415 TWh en 2024, equivalentes a cerca de 47.4 GW de demanda media lo cual se aproxima a 1.5% del consumo mundial de electricidad. Aunque esa cifra no se puede atribuir exclusivamente al contenido de video en línea, un estudio de Global Internet Phenomena de AppLogic confirmó que el streaming de video continúa siendo la principal fuente de tráfico en internet, con 38% del total global y que las aplicaciones de redes sociales de videos en línea y entretenimiento dominan el volumen de red por internet.

Estos datos, demuestran el estado actual de las plataformas de streaming de video bajo demanda, el alcance masivo que tienen y su elevadísima frecuencia de uso. No son solo aplicaciones de entretenimiento sino, servicios digitales con infraestructuras extremadamente complejas capaces de concentrar a miles de millones de personas, acumular millones de horas de video y moldear los hábitos de entretenimiento de la sociedad.

No obstante, cuanto más dominantes se han vuelto estas plataformas, más visibles se han hecho las quejas y los descontentos de sus usuarios.

En cuanto al sesgo, las cámaras de eco y la sensación de encierro algorítmico, los datos son significativos. Ofcom [6] encontró que alrededor de un tercio de los usuarios manifiesta incomodidad explícita con los algoritmos de recomendación y que, a su vez, los algoritmos no suelen favorecer las opiniones con las que discrepa un usuario.

Con respecto al contenido producido por IA, contenido inapropiado y desinformación, la escala del malestar es incluso más apreciable. Ofcom [7] reportó que 41% de los adultos usuarios de internet en el Reino Unido encontró desinformación en las cuatro semanas previas a la encuesta, 22% vio imágenes o videos falsos o engañosos, y 25% dijo haber quedado realmente molesto u ofendido por el daño más reciente encontrado online. Además, 47% nunca había visto herramientas, etiquetas o íconos que le ayudaran a entender si un contenido había sido creado o editado con IA, aunque 85% consideró importante que las plataformas lo informaran.

Asimismo, un estudio académico publicado en European Journal of Operational Research en 2025 [8], demuestra un malestar significativo de los usuarios frente a la sobrecarga publicitaria en redes sociales de video. Tomando datos globales sobre uso de bloqueadores de anuncios, señala que las razones más frecuentes para bloquear anuncios son la cantidad excesiva de anuncios (62.1%) y su carácter intrusivo (54.2%), lo que constituye una señal clara de fatiga publicitaria en entornos digitales de streaming de video. A ello se suma que el estudio global Media Reactions 2025 de Kantar muestra que la percepción de una plataforma mejora cuando sus anuncios son vistos como menos intrusivos, como ocurrió con Snapchat. Esto confirma que la intrusividad publicitaria es un criterio central de evaluación para los usuarios de plataformas sociales de video y un motivo de malestar muy frecuente para los mismos.

Inspirado en la necesidad de resolver estos problemas entendiendo que existe una oportunidad de mercado, nace la idea de *Booster* como una plataforma de videos alternativa cuyo algoritmo particular y enfoque a la experiencia de usuario permita minimizar los problemas presentados. En definitiva, *Booster* no se plantea, como una réplica de las plataformas existentes, sino como una respuesta tecnológica a necesidades no resueltas del mercado, con el fin de ofrecer un entorno, más equilibrado y más confiable tanto para quienes consumen contenido como para quienes lo producen

1.2. Objetivos

El objetivo de *Booster* es diseñar y desarrollar una plataforma de video web sustentada en un algoritmo de recomendación impulsado por la participación activa de los usuarios en la plataforma, nuevos métodos de moderación, una disminución de la cantidad de anuncios y la posibilidad de ver anuncios voluntarios, una experiencia de usuario basada en métodos de gamificación y la habilidad de crear y encontrar comunidades de usuarios y contenido más fácilmente.

De esta forma, *Booster* está orientado a responder a una necesidad creciente del servicios de video bajo demanda, con el objetivo de ofrecer a creadores y usuarios un entorno más transparente, y más eficiente que las plataformas dominantes actuales.

Esta necesidad surge de la acumulación de problemas que afectan tanto la experiencia de consumo como la de la creación de contenidos, entre ellos las dificultades de monetización para los creadores, la opacidad algorítmica en la distribución de contenidos, la formación de cámaras de eco, la reproducción de sesgos en los sistemas de recomendación, la abundancia de contenido generado por inteligencia artificial sin mecanismos suficientes de identificación y la percepción de censura injustificada derivada de procesos de moderación poco claros. En este contexto, *Booster* se propone como una solución tecnológica capaz de abordar dichos problemas mediante un algoritmo que optimice la personalización y el descubrimiento de contenido.

En consecuencia, *Booster* no se plantea únicamente como una nueva plataforma de distribución audiovisual, sino como una propuesta de arquitectura digital orientada a resolver necesidades reales del mercado.

Para desarrollar esta plataforma, se implementará una página web ya que de esta forma se puede llegar a un público más amplio y se puede implementar tecnologías modernas que faciliten el desarrollo. Además, se implementará un sistema de procesamiento y transcodificación de videos que permitirá a los usuarios subir sus propios videos y que estos puedan ser reproducidos en la plataforma.

1.3. Estructura de la memoria

El trabajo fin de grado se organiza en una secuencia de capítulos que conduce al lector desde la comprensión del problema hasta la formulación, diseño y

validación de la solución propuesta.

- **Capítulo 1 - Introducción:** Este capítulo presenta el contexto del trabajo, el estado actual de las plataformas de video bajo demanda, los problemas que enfrentan los usuarios y creadores, y los objetivos que se persiguen con el desarrollo de *Booster*.
- **Capítulo 2 - Estado del Arte:** En este capítulo se realiza un análisis detallado de las plataformas de video bajo demanda existentes, prestando atención al dominante YouTube y a otras alternativas como Rumble, Odysee y Peertube. Se pretende identificar las fortalezas y debilidades de cada una, las tecnologías que emplean, y las oportunidades de mejora.
- **Capítulo 3 - Tecnologías Estudiadas:** En este capítulo se detalla el análisis tecnológico que fundamenta al proyecto *Booster*. Se describen las principales tecnologías web seleccionadas y las descartadas para el objetivo del proyecto, un análisis de proveedores de servicios, elecciones de terceros. La finalidad es mostrar que las decisiones técnicas adoptadas responden a criterios de escalabilidad y usabilidad.
- **Capítulo 4 - Requisitos del Sistema:** En este capítulo se indican los requisitos funcionales y no funcionales de *Booster*. Se especifican las capacidades mínimas que ofrece la plataforma, las condiciones de rendimiento esperadas, las restricciones del sistema y las necesidades de usuarios y creadores.
- **Capítulo 5 - Innovaciones:** Se describen las innovaciones tecnológicas, de diseño y de experiencia de usuario que se han implementado para solventar los problemas identificados. Se detallan los aspectos novedosos de la plataforma y su factor diferencial frente a las alternativas existentes.
- **Capítulo 6 - Diseño y Arquitectura:** En este capítulo se presenta la arquitectura general de *Booster* y el de procesamiento y transcodificación de vídeos, los componentes principales del sistema, su interacción y el diseño de la interfaz de usuario. Aquí se explican, entre otros elementos, el algoritmo de recomendación, los sistemas de interacción como comentarios y XP, la moderación de contenidos y la experiencia general de usuario, búsqueda semántica de videos.
- **Capítulo 7 - Desarrollo:** Se detalla la implementación de ciertos componentes de la aplicación y su integración, los retos técnicos enfrentados y las soluciones adoptadas.
- **Capítulo 8 - Pruebas, Validación y Riesgos de Seguridad Detectados:** Este capítulo recoge las pruebas realizadas sobre la plataforma, tanto desde el punto de vista funcional como de seguridad, desempeño y experiencia de usuario. Se detallan medidas de seguridad tomadas, ataques reales detectados y mitigados, y se presentan los resultados de las pruebas.
- **Capítulo 9 - Conclusiones:** Se sintetizan los principales resultados del trabajo, evaluando qué se ha conseguido con el desarrollo de *Booster* según

los objetivos planteados. Asimismo, se valoran las aportaciones del proyecto, sus limitaciones y las posibles mejora o ampliaciones de la plataforma.

- **Capítulo 10 - Análisis de Impacto:** Se realiza un análisis del impacto del proyecto, considerando sus posibles implicaciones tecnológicas, económicas y medioambientales.

Capítulo 2

Estado del Arte

Este capítulo constituye una recopilación de diversas tecnologías disponibles para implementar un servicio web de streaming de videos. Asimismo, se exploran distintos modelos de arquitectura empleados por las aplicaciones que ya brindan este servicio.

2.1. Procesamiento, Transcodificación y Entrega de Video

En esta sección se hace un análisis de las herramientas más comunes para procesar, almacenar, transcodificar y entregar los videos en la plataforma.

2.1.1. FFmpeg

FFmpeg es una herramienta de código abierto utilizada para el procesamiento de contenido multimedia, en particular audio y video. Es empleada en una amplia variedad de tareas y aplicaciones como:

- **Transcodificación:** FFmpeg puede convertir videos entre distintos formato, códecs ¹ y resoluciones, lo que es esencial para garantizar la compatibilidad con diferentes dispositivos y plataformas. Por ejemplo, se puede convertir un video en formato *.mov* a *.mp4*, o extraer el audio de un archivo *.mp4* a *.mp3*.
- **Compresión y Optimización:** Permite reducir el tamaño de archivos sin perder calidad significativa ajustando la resolución, códec y el bitrate ². Esto es esencial para mejorar la experiencia de usuario. Por ejemplo, en este tipo de servicios se suele utilizar **Adaptive Bitrate**. Esto consiste en guardar varias copias del mismo video con diferentes calidades y tamaños

¹Se entiende por códec a aquella tecnología software o hardware que permite comprimir o descomprimir archivos multimedia.

²Se entiende por bitrate (tasa de bits) como la cantidad de información que se transmite por unidad de tiempo.

y, de esta forma, el reproductor puede seleccionar la versión más adecuada según la velocidad de conexión del usuario.

- **Edición de video:** Permite realizar tareas básicas de edición como recortar la duración, unir varios videos, rotar, etc.
- **Streaming:** FFmpeg puede ser utilizado para transmitir video en tiempo real a través de protocolos como RTP o UDP.



Figura 2.1: Logo de FFmpeg

Ventajas

La principal ventaja de FFmpeg es su versatilidad y flexibilidad. Puede aplicar filtros complejos, producir salidas en distintos formatos, ajustar la resolución, bitrate, códecs, frame rate y parámetros de la codificación dentro del mismo flujo de trabajo. Eso lo vuelve muy útil para transcodificación, remuxing, extracción de audio, recortes, subtítulo, entre otros casos de uso en el procesamiento de video.

Es una herramienta *open source* lo que implica que reduce la dependencia de un tercero, está disponible gratuitamente y está bien documentada. Todo esto facilita su integración en cualquier tipo de proyecto, evitando así el *vendor lock-in*.

La herramienta destaca por ser veloz y eficiente, implementando algoritmos avanzados de procesamiento de video. Además, es compatible con una amplia variedad de formatos y códecs y tiene soporte para diversos sistemas operativos. Por estas razones, FFmpeg es una opción popular y altamente utilizada en la industria para el procesamiento de video.

Desventajas

La principal desventaja de la herramienta es su complejidad. Requiere un conocimiento técnico profundo para su uso efectivo. A esto, se le suma su sintaxis compleja y la gran cantidad de opciones disponibles, lo que puede resultar abrumador para los usuarios sin experiencia.

FFmpeg es el ganador en términos de flexibilidad y control, pero sin embargo, cuando el objetivo es la velocidad extrema o el ahorro masivo de energía, el hardware dedicado o las soluciones implementadas desde cero pueden superar a FFmpeg. Por ejemplo, el chip personalizado de Google Argos VCU (Video Coding Unit) supera a FFmpeg en velocidad, eficiencia de cómputo y ahorro energético optimizando la capacidad de los centros de datos de YouTube. [9]

Servicios que emplean FFmpeg

- Netflix, que utiliza esta herramienta para la transcodificación de sus videos.
- Youtube, que incorpora FFmpeg y tecnologías personalizadas basadas en FFmpeg para procesar contenido multimedia.
- Meta, que también emplea FFmpeg para la transcodificación en plataformas como Facebook e Instagram.
- TikTok, que también utiliza FFmpeg para procesar los videos junto con tecnologías personalizadas.
- Nasa, utiliza FFmpeg en algunos de sus rovers marcianos para comprimir los videos capturados por las cámaras de estos robots antes de ser enviados a la Tierra.

2.1.2. MPEG-DASH

MPEG-DASH [10] (Dynamic Adaptive Streaming over HTTP) es un estándar internacional de transmisión de video sobre HTTP, enfocado en una distribución eficiente definido en la familia **ISO/IEC 23009**. Para ello, implementa técnicas de **Adaptive Bitrate** las cuales permiten ajustar automáticamente la calidad de reproducción, cambiando parámetros como el bitrate y la resolución del video según las condiciones de red y las capacidades del dispositivo del usuario. De esta forma, se garantiza una experiencia de visualización fluida y sin interrupciones, incluso en conexiones lentas e inestables.

Ventajas

La principal ventaja de MPEG-DASH es su escalabilidad y eficiencia. Al hacer uso de HTTP, se puede aprovechar la infraestructura existente de servidores web y CDNs para distribuir el contenido eficazmente a una audiencia global. Además, al implementar Adaptive Bitrate el video se divide en segmentos pequeños y se reduce la latencia.

Asimismo, MPEG-DASH es ampliamente compatible con una variedad de códecs, formatos de videos y dispositivos, lo que lo hace una opción versátil para la distribución de contenido multimedia.

Desventajas

MPEG-DASH, puede ser complejo de implementar y configurar. DASH, suele exigir una configuración cuidadosa de los servidores y los reproductores de video.

Aunque DASH funciona bien para la transmisión de video, no es la mejor opción para escenarios donde se requiere una latencia sumamente baja. En tales casos, se puede optar por utilizar Low-Latency DASH (LL-DASH) u otros protocolos alternativos como WebRTC.

Otra desventaja importante es que este protocolo no tiene soporte nativo en Apple, lo que limita su incorporación en este tipo de dispositivos.

Servicios que implementan DASH

DASH es un estándar abierto y empleado en gran cantidad de servicios de streaming. Algunos de estos son:

- YouTube es uno de los casos más importantes. Google informa a los desarrolladores que YouTube utiliza MPEG-DASH en su plataforma HTML5 para implementar Adaptive Bitrate.
- Brightcove, una empresa proveedora de servicios de software en la nube especializada en el alojamiento, distribución y gestión de video online, también ha informado que utiliza MPEG-DASH para la entrega de contenido multimedia.
- Bitmovin, otro proveedor de infraestructura de streaming, también ha confirmado que emplea MPEG-DASH en su plataforma para la entrega de video,.
- Vimeo, una plataforma de alojamiento de videos similar a YouTube, también ha confirmado que utiliza MPEG-DASH e implementa Adaptive Bitrate.
- Odysee y Peertube, plataformas de videos descentralizadas y basadas en tecnologías blockchain, adoptan también MPEG-DASH para la entrega de contenido multimedia.

2.1.3. HLS

De forma similar a MPEG-DASH, **HLS** [11] (HTTP Live Streaming) también es un protocolo de transmisión de video desarrollado por Apple. HLS implementa técnicas de **Adaptive Bitrate** y es utilizado para la distribución de contenido audiovisual bajo demanda y en directo fundamentalmente en dispositivos Apple, aunque es compatible con otros sistemas operativos.

Ventajas

Usualmente, este protocolo divide el video en segmentos pequeños y en formato de archivo `.m3u8` que permite entregar el contenido de forma eficiente por HTTP usando CDNs y servidores web. El reproductor de video solicita los segmentos más adecuados teniendo en cuenta la calidad de conexión del usuario mejorando la experiencia de visualización.

Además, HLS es altamente escalable. Esto lo hace ideal para la distribución de contenido a gran escala siendo una opción popular para servicios de streaming en vivo y bajo demanda con momentos de elevado tráfico.

HLS, es compatible con una amplia variedad de dispositivos especialmente con los dispositivos Apple, lo que lo convierte en una opción preferida para la distribución de contenido a usuarios de iOS y macOS.

Desventajas

Una de las desventajas más notorias de este protocolo es la falta de soporte en algunos dispositivos y navegadores, especialmente en aquellos que no son de

2.1. Procesamiento, Transcodificación y Entrega de Video

Apple. Esto puede limitar su adopción en ciertos entornos y requerir la implementación de soluciones alternativas para garantizar la compatibilidad con una audiencia más amplia. [12]

Adicionalmente, HLS puede ser inadecuado para circunstancias en las que se requiera una latencia extremadamente baja, como en llamadas de video en tiempo real. Debido a su arquitectura, para este tipo de aplicaciones, otros protocolos como WebRTC podrían ser más apropiados. [13]

Servicios que implementan HLS

La gran mayoría de actores principales en el mercado de streaming implementan este protocolo. Este es el caso de:

- Apple, siendo el creador del protocolo e implementándolo en sus dispositivos y servicios de streaming.
- AWS, en su servicio de live streaming
- Mux, un proveedor de infraestructura de streaming que ofrece soporte para HLS.
- Cloudflare, que también ofrece servicios de streaming con soporte para HLS.

Cabe destacar que, en ambos protocolos HLS y MPEG-DASH, el video no se descarga por completo de una vez, sino que el reproductor va solicitando pequeños segmentos o fragmentos al servidor conforme avanza la reproducción. En otras palabras, el video está almacenado en partes pequeñas en el servidor cada una con diferentes calidades y el reproductor le solicita la parte que mejor se adapte a las condiciones de red del usuario en cada momento. Así, se evitan transmitir grandes cantidades de datos innecesarios por la red y se puede implementar un Adaptive Bitrate eficiente.

2.1.4. Object Storage

El contenido multimedia, en particular los videos, suelen ocupar un gran espacio de almacenamiento. Además, con las técnicas de **Adaptive Bitrate** es necesario almacenar varias copias del mismo video con diferentes calidades y tamaños lo cual incrementa el uso de espacio de forma considerable. Por ello, es fundamental contar con un sistema de almacenamiento eficiente y escalable. Por esa razón, se suelen emplear sistemas de almacenamiento de objetos **Object Storage** [14] como Google Cloud Storage, Amazon S3 Buckets o Azure Blob Storage que permiten almacenar grandes cantidades de datos de forma escalable y con alta disponibilidad. Esto es esencial para garantizar un acceso rápido y confiable a los videos por parte de los usuarios.

Ventajas

La mayor ventaja es su escalabilidad. Amazon S3 o Google Cloud Storage, son servicios de almacenamiento en la nube que permiten almacenar y recuperar

Capítulo 2. Estado del Arte

cantidades masivas de datos solucionándole al usuario complicaciones con la gestión de particiones, discos, entre otros.

Estos servicios suelen ofrecer una alta disponibilidad y durabilidad, lo que prácticamente garantiza que los datos estarán siempre accesibles y protegidos contra pérdidas.

Desventajas

Aunque son capaces de almacenar grandes cantidades de datos, el acceso a los mismos no es tan rápido. Los Object Storage no están pensados para accesos con latencias extremadamente bajas. Por esa razón, suelen ser utilizados en conjunto con CDNs que permiten entregar el contenido de forma más eficiente a los usuarios finales.

Servicios de Object Storage

Todos las grandes plataformas de streaming emplean Object Storage para almacenar los videos. Los servicios más comunes para implementar esta tecnología son:

- Amazon S3 Buckets, es el ejemplo más conocido y es extensivamente usado por empresas y servicios como Netflix, Disney+, Twitch, Hulu y Prime Video para almacenar su contenido.
- Google Cloud Storage, es otro servicios de Object Storage empleado por YouTube, Spotify, Vimeo, entre otros.
- Azure Blob Storage, otro servicio de Object Storage utilizado por BBC Player, NBA, Real Madrid, entre otros.

2.1.5. CDN - Content Delivery Network

Un Content Delivery Network (CDN) [15] es una red de servidores distribuidos geográficamente que trabajan juntos para entregar contenido de manera rápida y eficiente a los usuarios finales. Un usuario, al solicitar un video, es atendido por el servidor más cercano a su ubicación, con el objetivo de reducir la latencia, aumentar la escalabilidad y mejorar el rendimiento del servicio. Su uso, se vuelve necesario en plataformas con alta demanda y con usuarios distribuidos geográficamente, donde depender exclusivamente del servidor de origen generaría mayores tiempos de respuesta, mayores costos y una sobrecarga innecesaria sobre la infraestructura principal.

De este modo, las CDN funcionan como una caché de contenido de los Object Storage, replicando o almacenando temporalmente los archivos en distintos nodos de la red para servirlos de manera más eficiente a los usuarios finales. Por tanto, los Object Storage cumplen la función de almacenamiento central del contenido, mientras que la CDN asume la función de distribución acelerada.

2.1. Procesamiento, Transcodificación y Entrega de Video

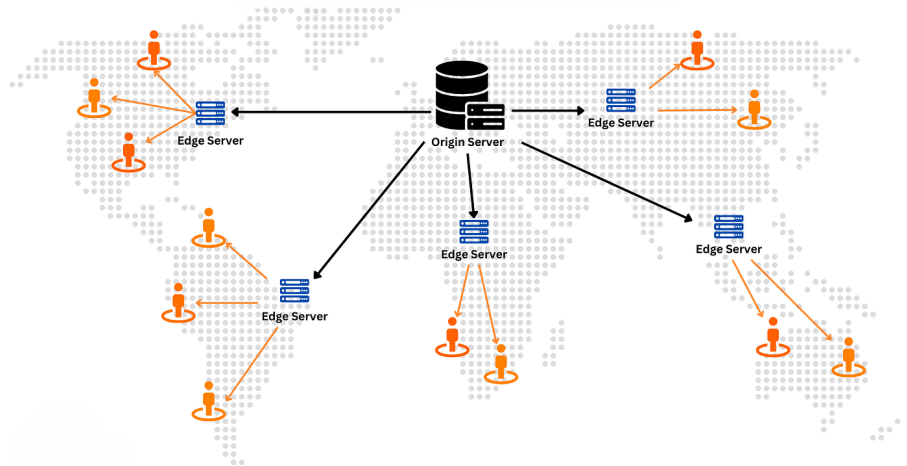


Figura 2.2: Diagrama de un CDN

Ventajas

Claramente, la principal ventaja de los CDNs es que reducen la latencia y mejora el rendimiento del servicio. Como los servidores están distribuidos geográficamente, el usuario accede al servidor más cercano y esto, reduce el tiempo de carga, mejorando la experiencia de usuario.

Además, el uso de CDNs reduce la carga en los servidores de origen y en los Object Storage. Al servir contenido desde los nodos de la CDN, el servidor principal recibe menos solicitudes directas, cumpliendo así una función de caché que alivia la infraestructura principal.

Los CDNs también facilitan la escalabilidad del servicio. En momentos de alta demanda, distintos CDNs pueden absorber el tráfico adicional permitiendo que el servicio se mantenga estable y sin depender exclusivamente del servidor de origen.

Por otra parte, un gran número de CDNs suelen ofrecer capacidades adicionales de seguridad. Esto incluye protección contra ataques DDoS, firewalls (WAF) y otras medidas de seguridad que ayudan a proteger el contenido y la infraestructura del servicio.

Desventajas

La desventaja más evidente es el costo. Aunque mejora la experiencia de usuario, facilita la escalabilidad y ofrece seguridad adicional, los CDNs suelen tener precios por transferencia, almacenamiento y solicitudes elevados, lo que puede resultar costoso para servicios con grandes volúmenes de tráfico y contenido.

Asimismo, los CDNs introducen complejidad adicional en la arquitectura del servicio. Se deben definir políticas de caché, gestionar la invalidación de contenido y configurar adecuadamente la distribución del contenido entre los distintos nodos. Incluso, puede existir la posibilidad de servir contenido obsoleto o incorrecto

si no se gestiona adecuadamente la sincronización entre el servidor de origen y los nodos de la CDN. Por tanto, el uso de CDNs requiera una gestión constante y una planificación rigurosa añadiendo complejidad a la arquitectura.

Servicios que emplean CDNs

A pesar de sus desventajas, el uso de CDNs es muy común en la industria, especialmente en el servicio de streaming. Normalmente, cada servicio utiliza sus propios CDNs para evitar la dependencia de un proveedor y ahorrar costos. Sin embargo, algunos de los servicios de CDNs más populares son:

- Cloudflare CDN, es un servicio de CDN utilizado por empresas como HyperConnect para el streaming de video y comunicaciones en tiempo real a escala global.
- Amazon CloudFront, es otra opción popular para entregar sitios web, APIs, archivos y streaming de videos globalmente.

2.1.6. Colas de Procesamiento

El procesamiento y la transcodificación de video son tareas que suelen tomar un largo tiempo en completarse. Por esa razón, cuando un usuario sube un video, en lugar de forzarlo a esperar, se utilizan colas de procesamiento. En vez de hacer las operaciones lentas en la misma petición HTTP, el sistema registra un trabajo pendiente en una cola y este espera a ser atendido por un worker. A continuación, se realiza este trabajo y se reportan los resultados al finalizar. Así, se consiguen desacoplar procesos lentos de otros más rápidos que de lo contrario, causarían una mala experiencia de usuario.

En el contexto de procesamiento de video el esquema sería el siguiente:

- 1. El usuario sube el video
- 2. El sistema guarda el archivo en un Object Store
- 3. Se crea un job en la cola de procesamiento
- 4. Un worker toma el trabajo pendiente
- 5. Se ejecuta la transcodificación de video, se generan copias del video, se comprime, etc.
- 6. Al terminar se notifica al sistema

Existen diversas formas de implementar una cola de procesamiento. Por ejemplo, RabbitMQ, Kafka, Amazon SQS (Simple Queue Service) suelen ser opciones comunes.

Ventajas

La ventaja principal es que mejoran la experiencia de usuario. La subida de video suele ser un proceso rápido mientras que la transcodificación puede tardar minutos e incluso horas en finalizar.

Además, las colas de procesamiento facilitan la escalabilidad. Este modelo permite responder de forma efectiva ante picos de alta demanda sin colapsar el backend: los trabajos permanecen en un estado de espera en la cola. Para evitar que haya saturaciones o una gran cantidad de trabajos pendientes en ser atendidos, una solución común es añadir más workers según sea necesario.

Por otra parte, también favorecen la priorización de trabajos, por ejemplo, dando paso a videos Premium o urgentes antes que el resto.

Desventajas

La desventaja más clara es que las colas de prioridad añaden complejidad a la arquitectura. Esto quiere decir que además de operar con las APIs y las bases de datos, ahora es necesario gestionar workers, reintentos, resolver duplicados, timeouts, entre otros.

Servicios de Colas de Procesamiento

Cada servicio de streaming suele implementar su propia solución de colas de procesamiento adaptada a sus necesidades. Sin embargo, existen soluciones comunes como:

- Amazon SQS, es una cola de procesamiento general ofrecida por Amazon y utilizada por servicios como Fox Sports u otros clientes.
- RabbitMQ, es otra opción popular utilizada por empresas de broadcasting de contenido como Norwegian Broadcasting Corporation (NRK).
- Apache Kafka, es una plataforma de streaming de eventos ampliamente utilizada por empresas como Netflix, LinkedIn, entre otros.

2.2. Análisis de Proveedores

En esta sección se hace un estudio de los principales proveedores de infraestructura de streaming que se han considerado e implementado en el proyecto. Se analizan sus características, ventajas, desventajas, costos, facilidad de implementación e integración, entre otros aspectos relevantes.

2.2.1. Amazon Web Services y solución propia

Esta alternativa consiste en implementar una solución propia utilizando los servicios de Amazon Web Services (AWS) para cada una de las etapas del proceso. Esto implica, implementar desde cero la lógica para el almacenamiento, entrega de video, transcodificación y procesamiento, generación de miniaturas y carga de videos al servidor.

En este proyecto, se ha implementado una solución propia utilizando AWS la cual será descrita más en detalle en el capítulo de arquitectura en donde se utilizan todas las tecnologías expuestas en la sección anterior. Esta solución, aunque es más compleja, permite un control total sobre el funcionamiento del servicio.

poner capítulo arquitectura y donde Explico Docker?

Además, es útil entender cómo los servicios proveedores de infraestructura de streaming como Mux o Bunny.net, implementan su solución.



Figura 2.3: Logo de AWS

Ventajas

La principal ventaja de utilizar un pipeline propio es que se tiene un control total sobre el funcionamiento del servicio. Esto permite ajustar cada etapa a las necesidades específicas del proyecto. Por ejemplo, se pueden ajustar los servicios de AWS para que se ajusten a la escala del proyecto, realizar el procesamiento de video de forma personalizada, entre otros.

A su vez, se evita la dependencia de un proveedor de infraestructura de streaming separado. Con esto se evita el *vendor lock-in* y se tiene la libertad de cambiar o ajustar los servicios de AWS según sea necesario sin depender de las limitaciones de un proveedor externo.

Asimismo, AWS es un servicio usado extensivamente en la industria, con una disponibilidad y escalabilidad probada.

Desventajas

Evidentemente, la principal desventaja es la complejidad que añade. Implementar esta solución implica gestionar y configurar cada uno de los servicios de AWS. Además, requiere saber sobre cada una de las etapas del proceso en detalle y su integración. Un proveedor externo suele resolver todos los problemas de mantenimiento, almacenamiento, colas, procesamiento, mantenimiento y seguridad, mientras que con esta solución, el equipo de desarrollo es el responsable de gestionar cada uno de estos aspectos. En consecuencia, los recursos y el tiempo de implementación son mayores.

Uno tendería a pensar que esta solución es más eficiente en términos de costos, pero no siempre es así, como se demostrará más adelante.

Servicios que emplean AWS

Algunos Servicios de streaming que emplean AWS o utilizan servicios de AWS para su infraestructura son:

- Netflix, para su infraestructura de streaming, utiliza AWS para almacenar y entregar su contenido multimedia.
- Amazon Prime Video, que es el servicio de streaming de Amazon, también utiliza AWS para su infraestructura de streaming.

2.2. Análisis de Proveedores

- Twitch, una plataforma de streaming de videojuegos, también utiliza AWS para su infraestructura de streaming.
- Vision+, un servicio de streaming de Indonesia, usa servicios de AWS.

Análisis de Costos

2.2.2. Mux.com

2.2.3. Bunny.net

Capítulo 3

Tecnologías Estudiadas

Capítulo 4

Requisitos del Sistema

Capítulo 5

Innovaciones

Capítulo 6

Diseño y Arquitectura

Capítulo 7

Desarrollo

Capítulo dedicado a describir el desarrollo del trabajo realizado. De acuerdo con el tutor, este capítulo puede tener distintas estructuras, e incluso pueden existir **varios capítulos**.

Por ejemplo, para un trabajo de desarrollo clásico, este capítulo de “Desarrollo” podría convertirse en tres capítulos:

- Requisitos (con la especificación de requisitos, ya sea clásico o “agile”)
- Análisis y diseño (con modelo de datos, arquitectura, diseño de experiencia de usuario e interfaz de usuario, etc.)
- Implementación (con detalles sobre la solución/implementación realizada).

Todos los capítulos deben empezar en una página nueva (esta plantilla ya lo hace automáticamente).

Los apartados dentro de los capítulos se numeran de forma jerárquica, pero siempre deben estar alineados al margen izquierdo. Ejemplo:

7.1. Apartado 1 de capítulo 2

7.1.1. Sección 1 de apartado 1 de capítulo 2

Sub sección 1

Párrafo 1 Con cierto contenido

Párrafo 2 Con más contenido

Sub sección 2

Párrafo 1 Con cierto contenido

Párrafo 2 Con más contenido

7.1.2. Sección 2 de apartado 1 de capítulo 2

7.2. Apartado 2 de capítulo 2

7.3. Apartado 3 de capítulo 2

Capítulo 8

Pruebas, Validación y Riesgos de Seguridad Detectados

Capítulo 9

Conclusiones

Resumen de resultados obtenidos en el trabajo, conclusiones sobre los mismos y potencial trabajo futuro si existiera.

Capítulo 10

Análisis de Impacto

En este capítulo se realizará un análisis del impacto potencial de los resultados obtenidos durante la realización del trabajo, en los diferentes contextos para los que se aplique:¹

- Personal
- Empresarial
- Social
- Económico
- Medioambiental
- Cultural

En dicho análisis se destacarán los beneficios esperados, así como también los posibles **efectos adversos**. Además, se harán notar aquellas decisiones tomadas a lo largo del trabajo que tienen como base la consideración del impacto.

Se recomienda analizar también el potencial impacto respecto a los Objetivos de Desarrollo Sostenible (ODS), de la Agenda 2030, que sean relevantes para el trabajo realizado (ver enlace)

¹Téngase en cuenta que no se espera que un trabajo tenga impacto en todos los contextos

Bibliografía

- [1] A. Inc. «Alphabet Announces Fourth Quarter and Fiscal Year 2025 Results», visitado 26 de mar. de 2026. dirección: https://s206.q4cdn.com/479360582/files/doc_financials/2025/q4/2025q4-alphabet-earnings-release.pdf
- [2] TikTok. «TikTok's community in Europe Tops 200 Million», visitado 26 de mar. de 2026. dirección: https://newsroom.tiktok.com/tiktok-community-in-europe-tops-200-million?from_seo_redirect=1&lang=en-150
- [3] I. Netflix. «FINAL Q4 25 Shareholder Letter», visitado 26 de mar. de 2026. dirección: https://s22.q4cdn.com/959853165/files/doc_financials/2025/q4/FINAL-Q4-25-Shareholder-Letter.pdf
- [4] I. Meta Platforms. «Meta Reports Fourth Quarter and Full Year 2025 Results», visitado 26 de mar. de 2026. dirección: <https://investor.atmeta.com/investor-news/press-release-details/2026/Meta-Reports-Fourth-Quarter-and-Full-Year-2025-Results/default.aspx>
- [5] International Energy Agency. «Energy and AI», visitado 26 de mar. de 2026. dirección: <https://www.iea.org/reports/energy-and-ai/energy-demand-from-ai>
- [6] Ofcom. «Adults' Media Use and Attitudes 2025», visitado 26 de mar. de 2026. dirección: <https://www.ofcom.org.uk/siteassets/resources/documents/research-and-data/media-literacy-research/adults/adults-media-use-and-attitudes-2025/adults-media-use-and-attitudes-report-2025.pdf?v=396240>
- [7] Ofcom. «Online Nations Report 2025», visitado 26 de mar. de 2026. dirección: <https://www.ofcom.org.uk/siteassets/resources/documents/research-and-data/online-research/online-nation/2025/online-nations-report-2025.pdf?v=409837>
- [8] X. Lu, X. Hou, N. Yuan y J. Wang. «Allow or not? Strategic choices of competing platforms in response to ad blockers», visitado 26 de mar. de 2026. dirección: <https://www.sciencedirect.com/science/article/abs/pii/S037872062500120X>

BIBLIOGRAFÍA

- [9] P. Ranganathan, D. Stodolsky, J. Calow et al. «Warehouse-Scale Video Acceleration: Co-design and Deployment in the Wild», visitado 28 de mar. de 2026. dirección: <https://gwern.net/doc/cs/hardware/2021-ranganathan.pdf>
- [10] MPEG. «MPEG-DASH», visitado 26 de mar. de 2026. dirección: <https://www.mpeg.org/standards/MPEG-DASH/>
- [11] Apple Inc. «HTTP Live Streaming», visitado 26 de mar. de 2026. dirección: <https://developer.apple.com/streaming/>
- [12] Mux Video. «Play your Videos | Mux», visitado 26 de mar. de 2026. dirección: <https://www.mux.com/docs/guides/play-your-videos>
- [13] H. P. G. T. F. Casper Haems Jeroen van der Hooft. «Hybrid Unicast–Broadcast Video Delivery for Scalable Low-Latency Live Streaming», visitado 26 de mar. de 2026. dirección: <https://dl.acm.org/doi/10.1145/3742868>
- [14] Google Cloud. «Google Cloud Storage», visitado 26 de mar. de 2026. dirección: <https://cloud.google.com/learn/what-is-object-storage>
- [15] Cloudflare. «What is a CDN?», visitado 26 de mar. de 2026. dirección: <https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>

Anexos

Apéndice A

Primer anexo

Este capítulo (anexo) es opcional, y se escribirá de acuerdo con las indicaciones del Tutor.